Applied Algorithms
Lecture 8: Route planning

October 22, 2025

(updated October 27)

Martin Aumüller

Status

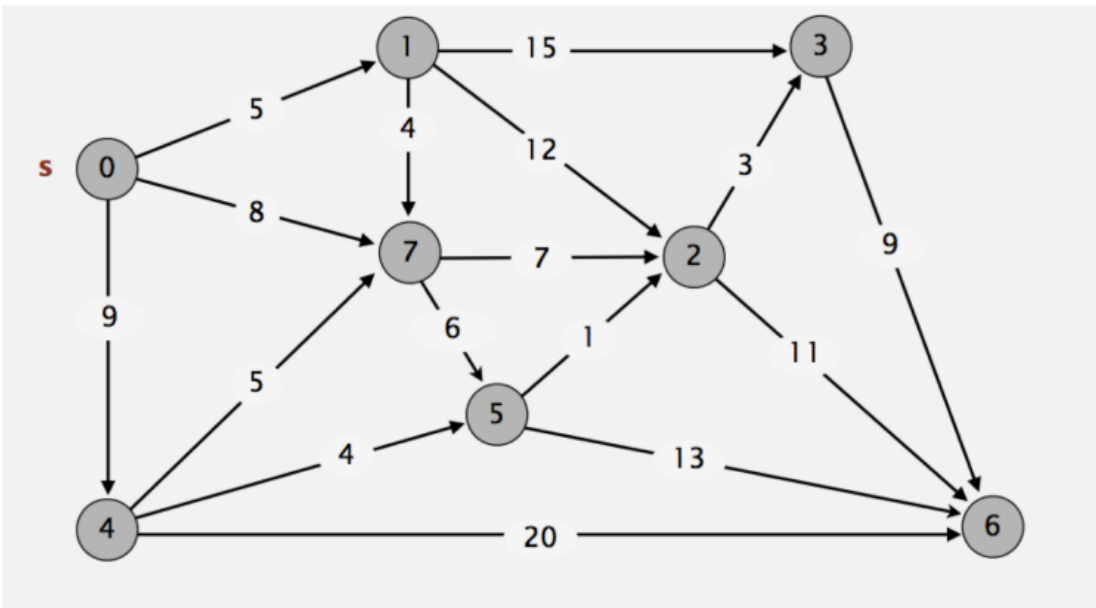
- Current focus: Algorithms
  - Fenwick trees, HyperLogLog, MinHash, Dynamic programming, …
  - Today: Routing in road networks
- Final assignment today, 3 weeks to complete
- Upcoming block: “General observations on Performance”
  - Cache hierarchy, parallelism, external memory
- Final project announced in 3 weeks

Topic for today



Problem Single-Source-Shortest Paths

- **Input**: A directed, weighted graph $G = (V, E, c)$ with weight function $c: E \to \mathbb{R}_{\geq 0}$.
- **Task**: For given $s \in V$, compute (the length of) the shortest paths to every $v \in V$. We denote the length of a shortest path between $s, v \in V$ by $d(s, v)$.
- **Example**:
  - $d(s, 5) = 13$
  - $d(s, 6) = 25$



Dijkstra's Algorithm

- Data structure for graph with vertices $\{1, \ldots, n\}$:
  - Arrays distTo[1..n] and edgeTo[1..n]
  - distTo[v] is length of a shortest path from source to v
  - edgeTo[v] is vertex of the last edge on a shortest path from source to v
- Basic operation: *relax*(v, w)
  - If distTo[w] > distTo[v] + $c(v, w)$

    Set distTo[w] = distTo[v] + $c(v, w)$
    edgeTo[w]=v

High-level description of Dijkstra's algorithm

1. Consider vertices in increasing order of distance from s (by looking at the distTo-value)
2. Add vertex to tree, relax all outgoing edges from that vertex

Demo

Implementation considerations (n vertices, m edges)

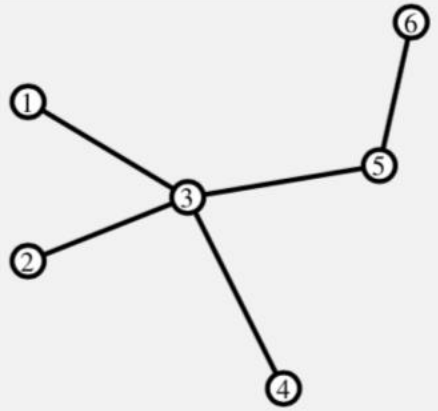
- **Question**: How to find the next node in the shortest path tree?
- **Solution 1**: Just search for smallest value in distTo array (as in demo)
  - Running time: $O(n^2)$
- **Solution 2**: Keep each node with its distTo value in a priority queue
  - Running time: $O(m \log n)$
  - ”eager”: each node is in the PQ at most once
  - PQ must support decreasing the priority of a key (Java’s PQ doesn’t)
- **Solution 3**: As solution 2, but ”lazy” variant
  - do not decrease, blindly insert
  - Running time: $O(m \log m) = O(m \log n)$ since $m \leq n^2$.

No difference asymptotically!

Where does the graph come from?

- Most popular source is openstreetmaps
- Format is XML
- Edge weights:
  - Distance between (lon, lat) pairs
  - Can take speed into consideration

```
<way>
    <nd ref="1">
    <nd ref="3">
    <nd ref="4">
</way>
<way>
    <nd ref="2">
    <nd ref="3">
    <nd ref="5">
    <nd ref="6">
</way>
```





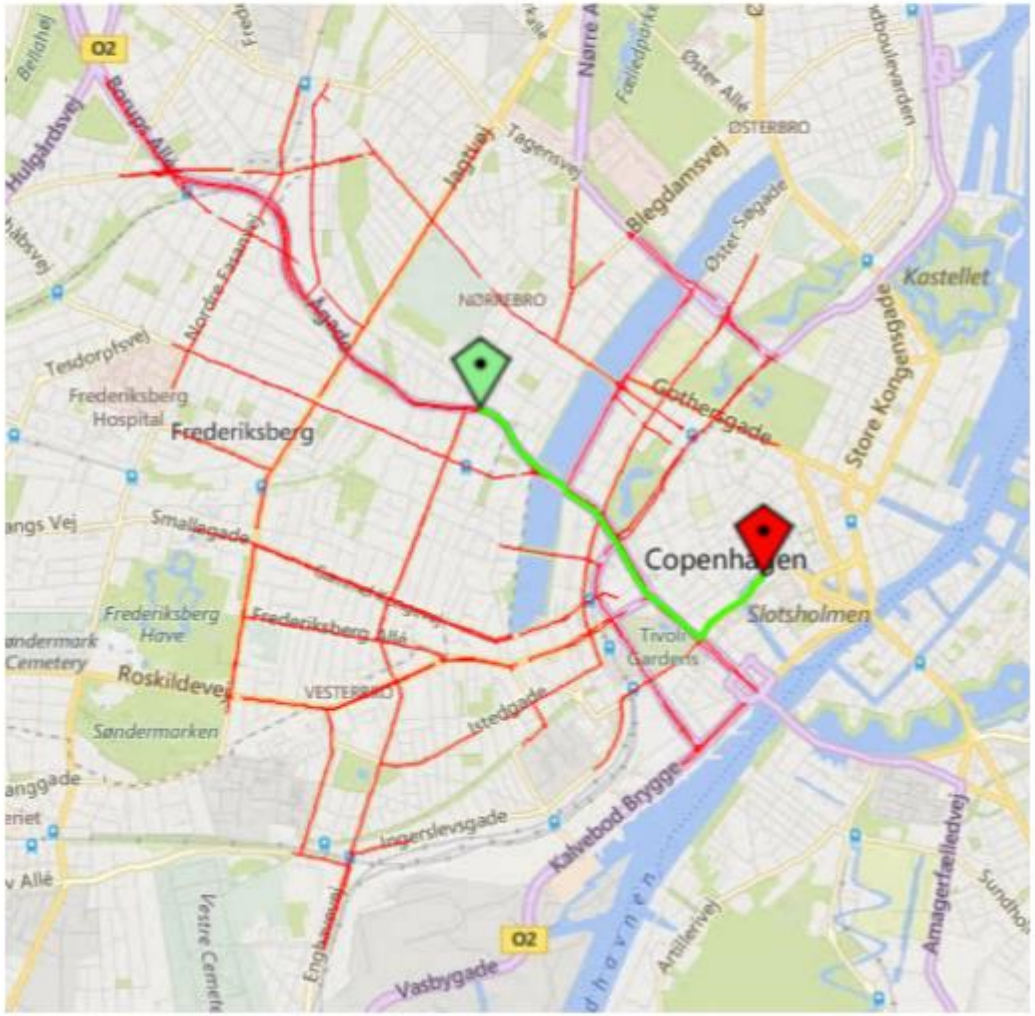
Denmark road network*: 570k vertices, 587k edges

(* Actually much larger, we did some preprocessing.)

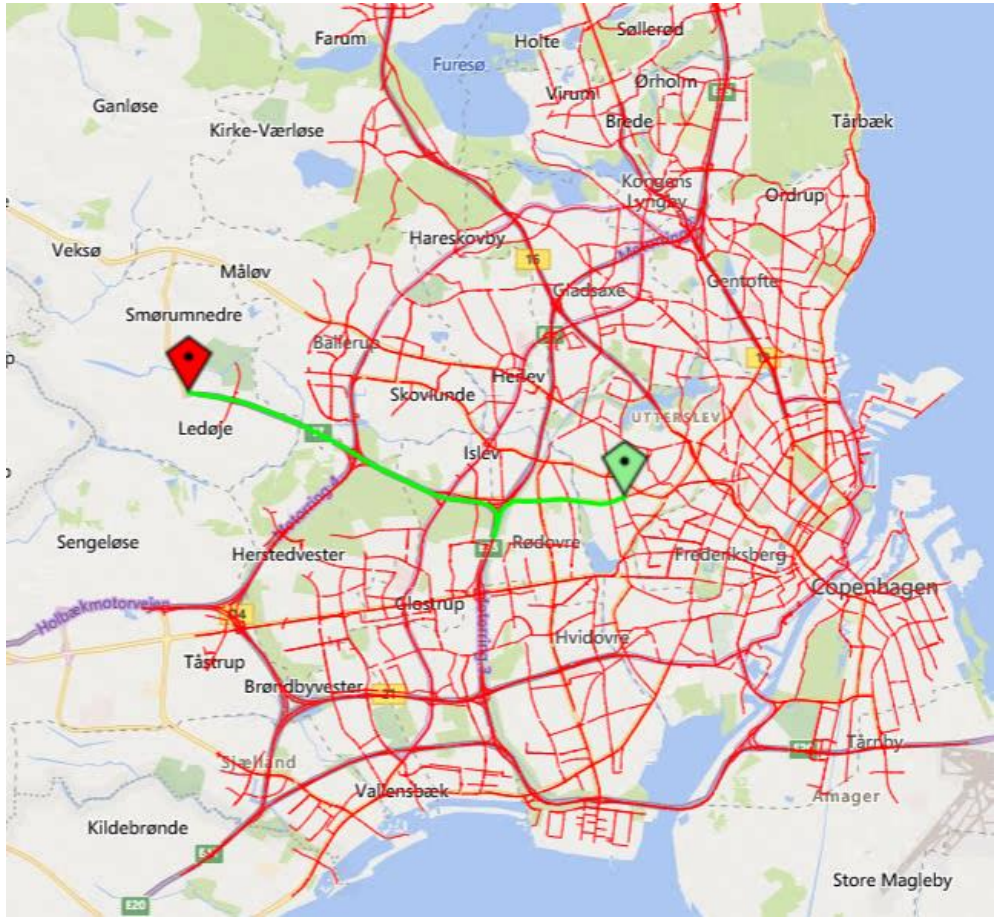
Improving Dijkstra

- Red: The shortest path tree
- Looks like a ”circle”, no orientation towards the goal

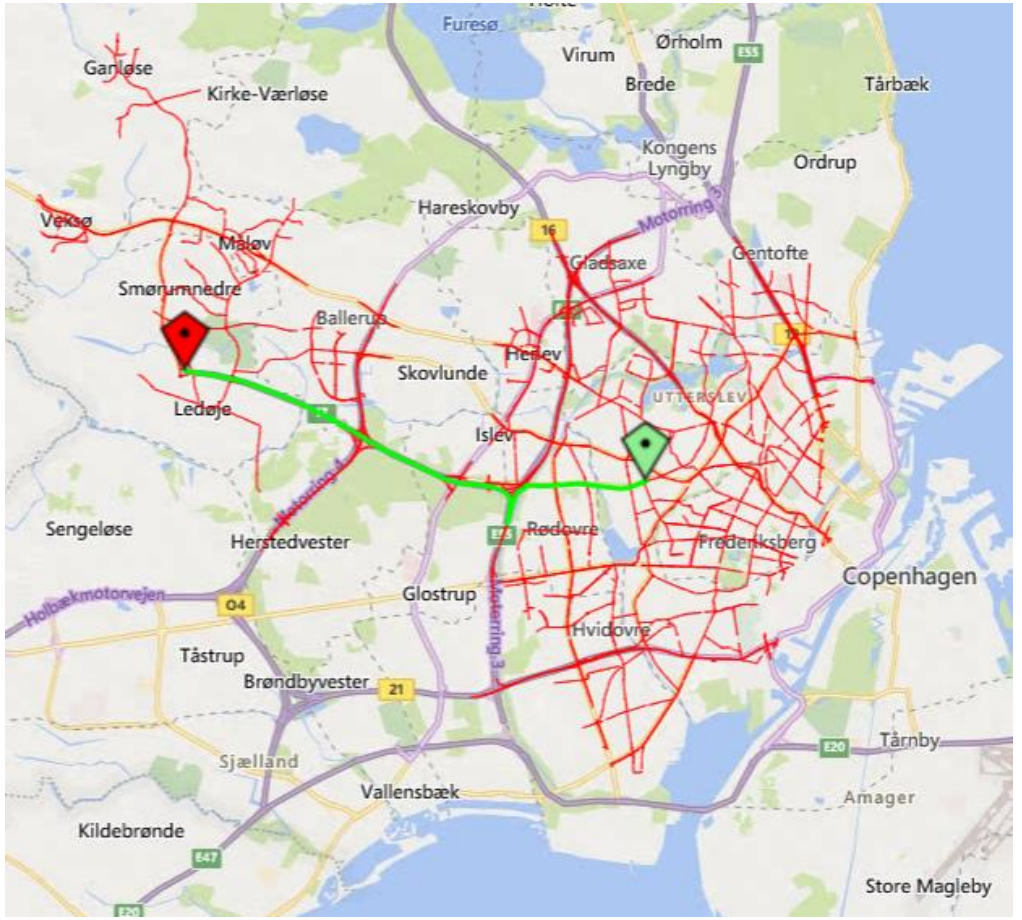
Modifying Dijkstra to answer shortest paths from s to t:
Carry out Dijkstra search,
stop as soon as node t is taken out of priority queue.



Bidirectional Dijkstra



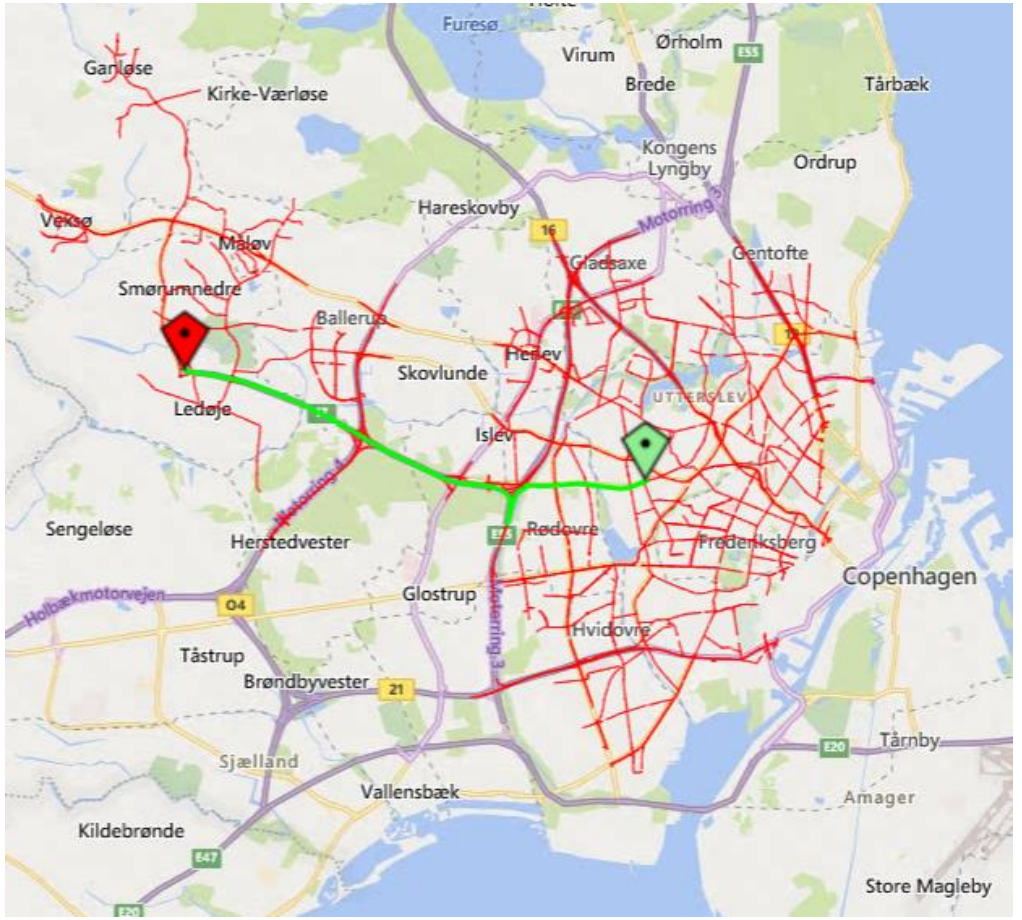
32713 edges in tree



13019 edges in tree

Outline of Bidirectional Dijkstra

- Run Dijkstra both from source (forwards) and target (backwards)
- Always consider the node at minimum distance in both runs
  - Do not alternate
- When do you stop?
  - Both runs consider same node?
- Always faster?
  - Not necessarily, could go from a sparsely populated area to border of dense area



Pseudocode Bidirectional Dijkstra

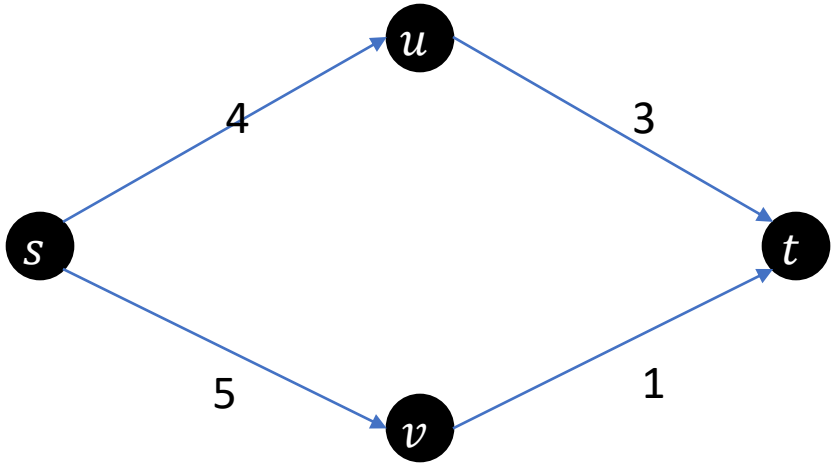
(See Assignment 3)

Algorithm 1 Bidirectional Dijkstra(s, t)

```
1: d_l ← ⟨∞, ..., ∞⟩, d_r ← ⟨∞, ..., ∞⟩, d ← ∞
2: settled ← ⟨False, ..., False⟩
3: d_l[s] ← 0, d_r[t] ← 0
4: Q_l ← {(s, 0)}, Q_r ← {(t, 0)} //PQs, l from start, r from target
5: while Q_l ≠ ∅ or Q_r ≠ ∅ do
6:    if Q_l ≠ ∅ and min(Q_l) ≤ min(Q_r) then
7:       i ← l
8:    else
9:       i ← r
10:   end if
11:   (u, ·) ← Q_i.deleteMin()
12:   if settled[u] then
13:      //This vertex was settled by the other instance
14:      break
15:   end if
16:   settled[u] ← True
17:   for all (u, v) ∈ E do
18:      if d_i[u] + c(u, v) < d_i[v] then
19:         d_i[v] ← d_i[u] + c(u, v)
20:         Update Q_i with (v, d_i[v])
21:      end if
22:      d ← min(d, d_l[v] + d_r[v]) //Update minimum dist between s and t
23:   end for
24: end while
25: return d
```

Stopping criterion:
Return shortest distance found until first node is settled by both

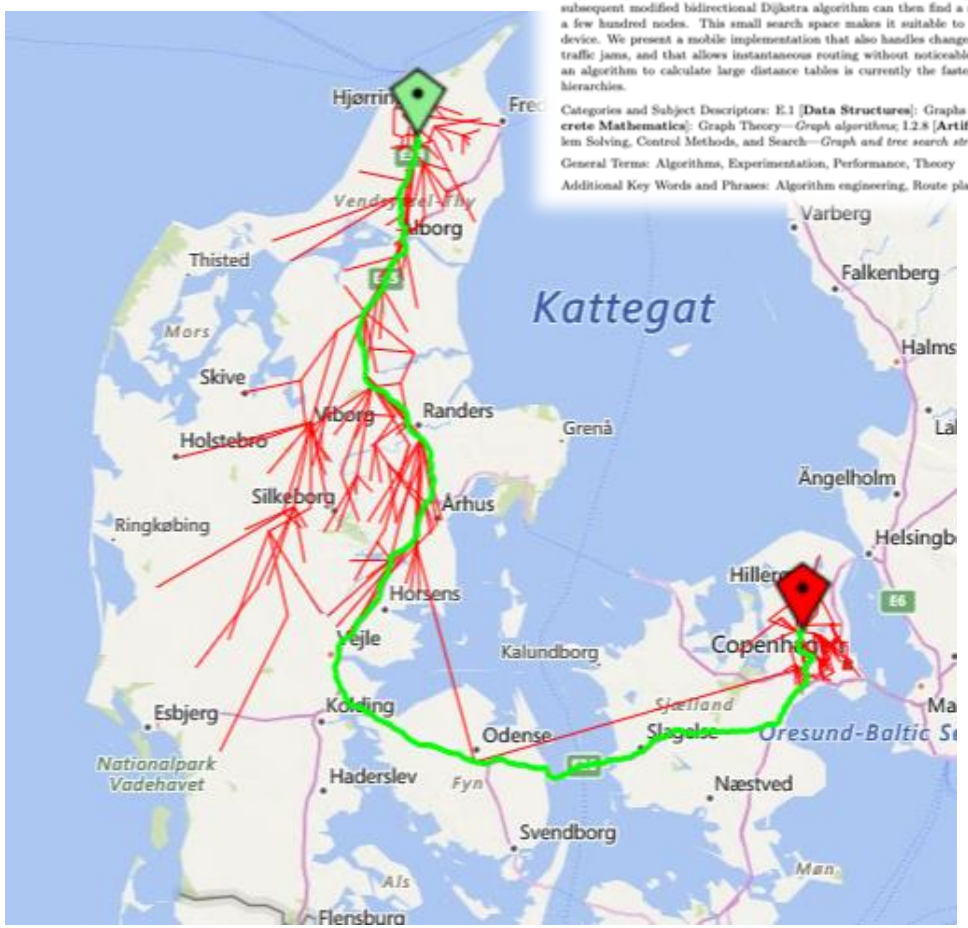
Check: u is first node settled by both, shortest path uses v



Efficient Route Planning

- Status: Dijkstra + modifications
  - Hundreds of milliseconds query time on map of Denmark
- But what about larger maps?
- Or many users accessing your service at the same time?

- Many ideas where floating around in the past, **Contraction Hierarchies** by [Geisberger et al., 2008] is a particularly beautiful solution



316 edges in tree

Exact Routing in Large Road Networks using Contraction Hierarchies

ROBERT GEISBERGER, PETER SANDERS, DOMINIK SCHULTES and CHRISTIAN VETTER
Karlsruhe Institute of Technology, 76128 Karlsruhe, Germany

Contraction hierarchies are a simple approach for routing in road networks. Our algorithm calculates exact shortest paths and handles road networks of whole continents. During a preprocessing step, we exploit the inherent hierarchical structure of road networks by adding shortcut edges. A subsequent modified bidirectional Dijkstra algorithm can then find a shortest path visiting only a few hundred nodes. This small search space makes it suitable to implement it on a mobile device. We present a mobile implementation that also handles changes in the road network, like traffic jams, and that allows instantaneous routing without noticeable delay for the user. Also, an algorithm to calculate large distance tables is currently the fastest if based on contraction hierarchies.

Categories and Subject Descriptors: E.1 [**Data Structures**]: Graphs and networks; G.2.2 [**Discrete Mathematics**]: Graph Theory—*Graph algorithms*; I.2.8 [**Artificial Intelligence**]: Problem Solving, Control Methods, and Search—*Graph and tree search strategies*

General Terms: Algorithms, Experimentation, Performance, Theory

Additional Key Words and Phrases: Algorithm engineering, Route planning, Shortest paths

Outlook

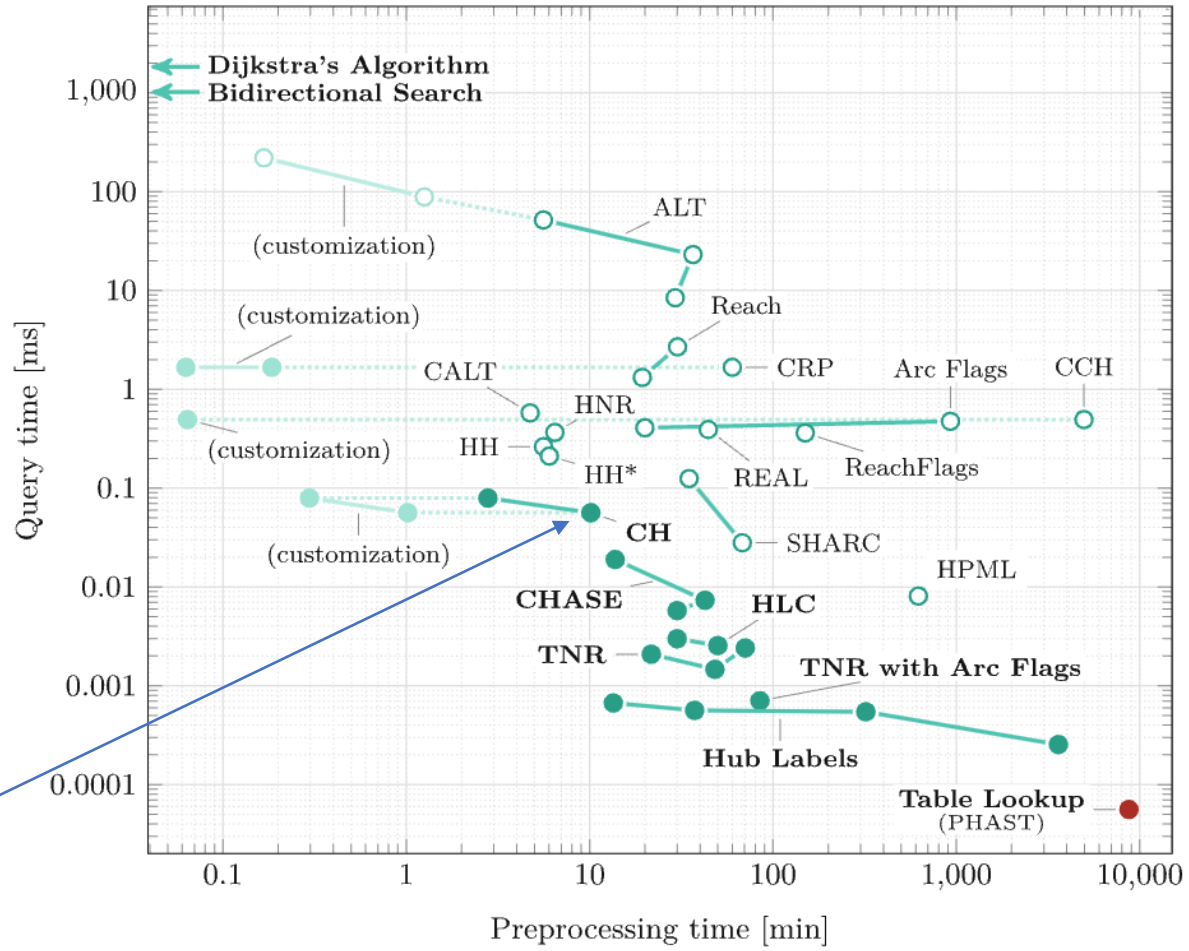


Fig. 7. Preprocessing and average query time performance for algorithms with available experimental data on the road network of Western Europe, using travel times as edge weights. Connecting lines indicate different trade-offs for the same algorithm. The figure is inspired by [238].

OUR FOCUS

[Bast et al., Route Planning in Transportation Networks, 2016]

Contraction Hierarchies Overview

- From input graph, construct *augmented graph*
  - Every node has a **rank**
  - New edges are created as **shortcuts**
- Augmented/Overlay graph is constructed by *iteratively removing vertices* and adding edges such that *shortest paths are preserved*
  - After i rounds, graph has $n - i$ vertices
- Bidirectional Dijkstra Search to find shortest paths

Preprocessing: Constructing an Overlay Graph

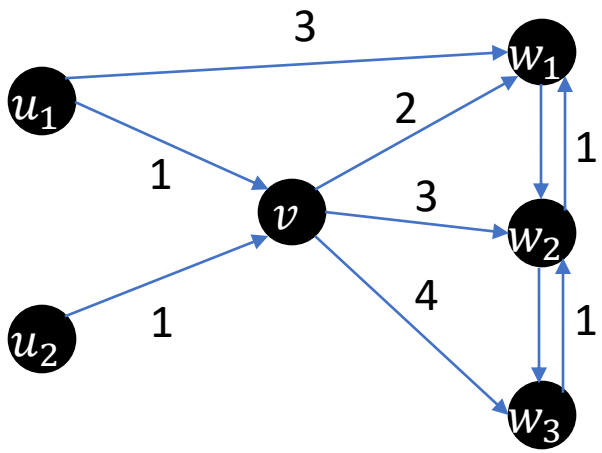
- Give each node in the input graph a unique rank according to its importance (many choices possible, but let it be arbitrary in the following)
- Assume $v_1, \ldots, v_n$ are sorted according to their rank, i.e.,
$$rank(v_1) \leq rank(v_2) \leq \ldots \leq rank(v_n)$$
- For each node v_i: contract(v_i)
- Contraction of a node v:

foreach *pair* (u, v) *and* (v, w) *of edges* **do**
  if (u, v, w) *is a unique shortest path* **then**
    add shortcut (u, w) with weight $w(u, v, w)$

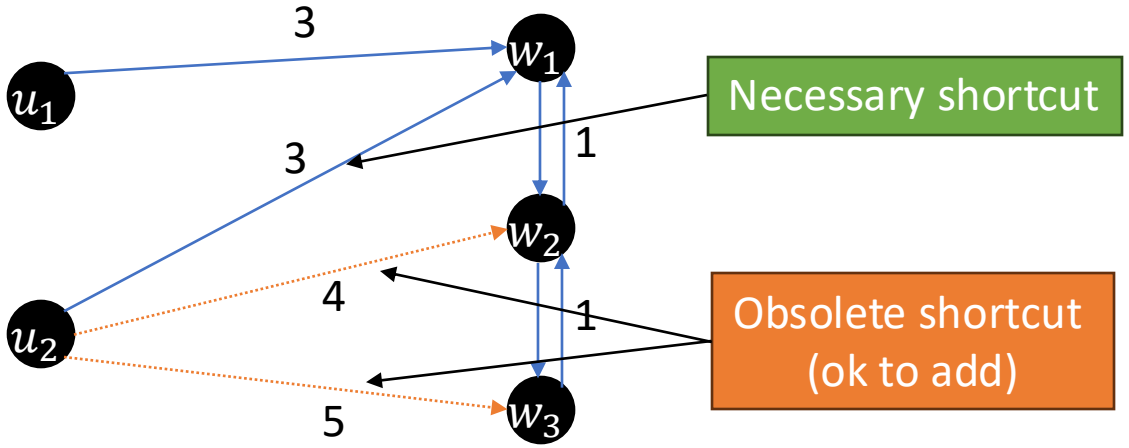
Example contraction on blackboard

(Ignoring ranks given to nodes)

Original graph before contract(v)



Resulting graph from contraction



Observations:

1. All shortest paths from u_1 are preserved without v.
2. Shortest paths from u_2 to w_i nodes is destroyed. Edge (u_2, w_1) with weight 3 preserves all shortest paths.
3. Might include additional shortcut edges due to ordering issues: For example, if (u_2, w_2) is inspected first, shortcut seems necessary until (u_2, w_1) is inspected. Disregarding dependencies simplifies implementation.

Larger example

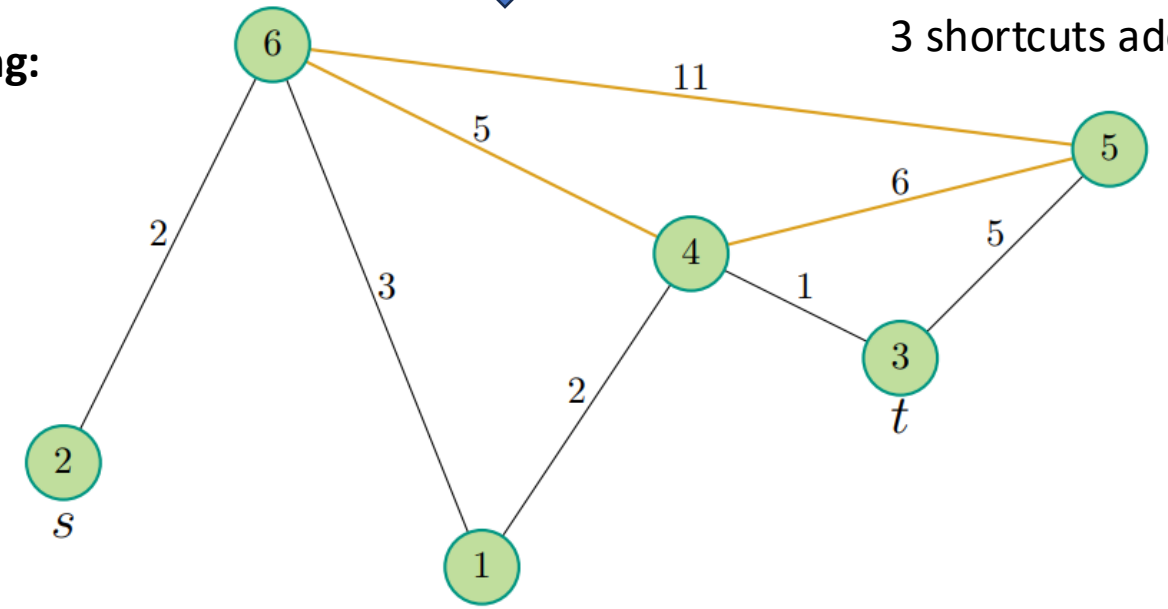
Check yourself: If ranks are 1, 2, 3, 4, 5, 6 (left-to-right), no shortcuts are added.



Data structure after preprocessing:

1. Original graph
2. Node ranks
3. All shortcuts added during contraction.

3 shortcuts added.



Example taken from https://i11www.iti.kit.edu/_media/teaching/winter2022/algorithmengineeringpraktikum/einfuehrung.pdf

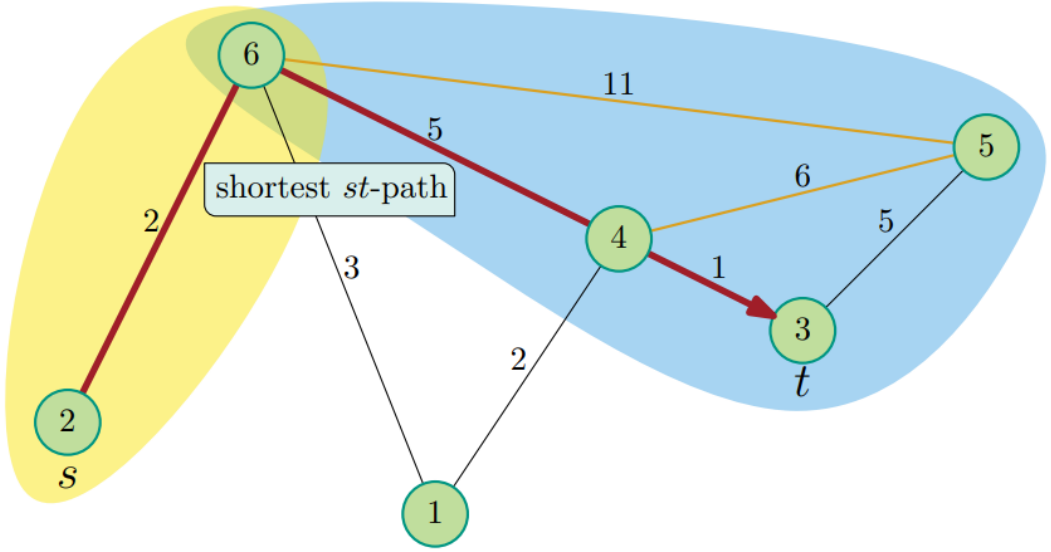
The catch

- Why would that ever be better?
  - We just added edges!
- Catch is in the query:
  - Bidirectional Dijkstra
  - From start,
    only going up in rank
  - From target (reversed),
    only going up in rank
  - **Termination:** both priority queues have priority larger than current best (s, t) path.
- **Found path:** Goes up in rank until some middle node, from which it only goes down.

Algorithm 1: Query(s,t)

```
1  d↑:= ⟨∞,...,∞⟩; d↓:= ⟨∞,...,∞⟩; d:= ∞;        // tentative distances
2  d↑[s]:= 0; d↓[t]:= 0;          // start bidirectional search from s and t
3  Q↑ = {(0,s)}; Q↓ = {(0,t)}; r:= ↑;                  // priority queues
4  while (Q↑ ≠ ∅ or Q↓ ≠ ∅) and (d > min{min Q↑, min Q↓}) do
5      if Q¬r ≠ ∅ then r:= ¬r;        // interleave direction, ¬↑=↓ and
         ¬↓=↑
6      (·,u):= Qr.deleteMin();                                  // settle u
7      d:= min{d, d↑[u] + d↓[u]};              // u is potential candidate
8      foreach e = (u,v) ∈ E* do                              // relax edges
9          if r(e) and (dr[u] + c(e) < dr[v]) then      // shorter path found
10             dr[v]:= dr[u] + c(e);              // update tentative distance
11             Qr.update(dr[v],v);                  // update priority queue
12 return d;
```

THEOREM 3.1. *Algorithm 1, applied to a CH, returns the correct shortest path distance.*



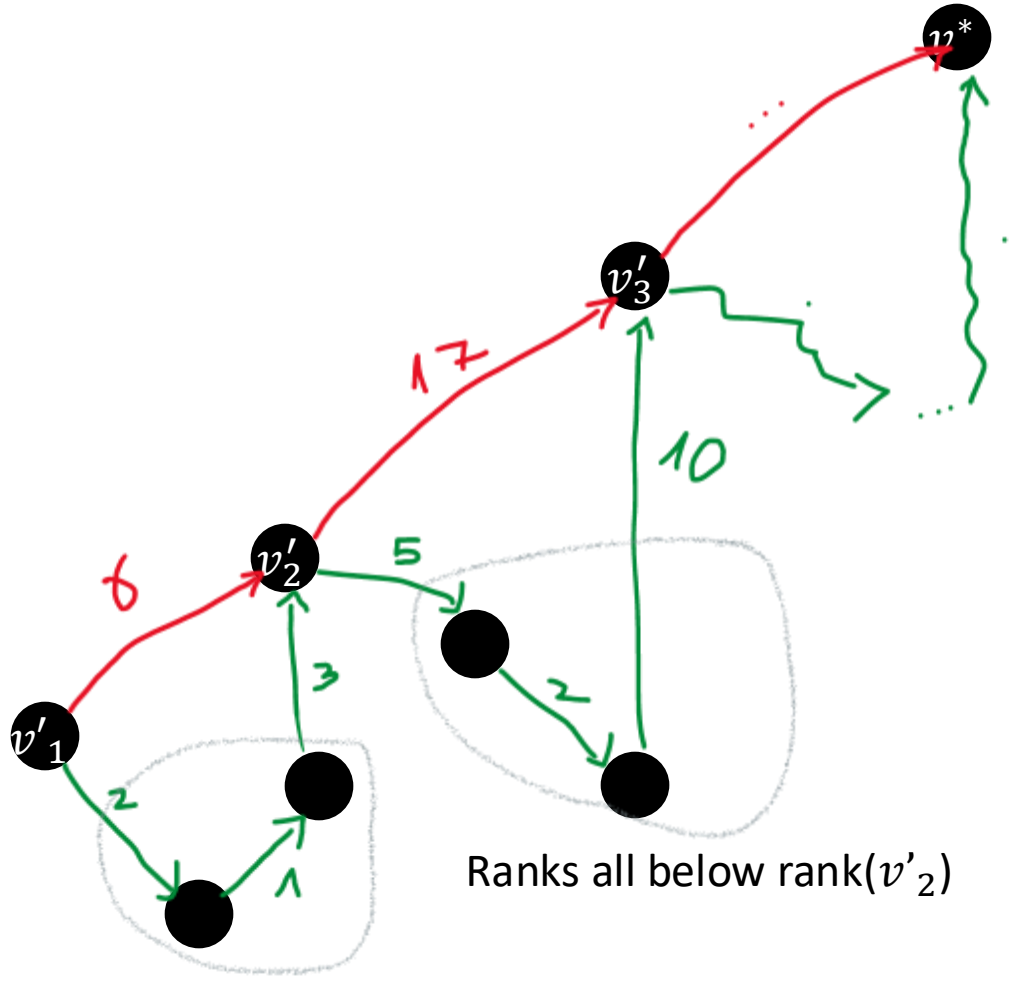
Idea of why this works

(only ”going up” part)

- Let $s = v_1, v_2, \dots, v^*, \dots, v_\ell = t$ be a shortest path p between s and t in graph.
- Let v^* be node of maximum rank, and let $v'_1, v'_2, \dots, v^*$ be the part of p such that rank(v'_1) < rank(v'_2) < … < rank(v^*) such that nodes between v'_{i-1} and v'_i all have rank lower than rank(v'_{i-1}).
- Proof: All red edges must exist.

Proof idea

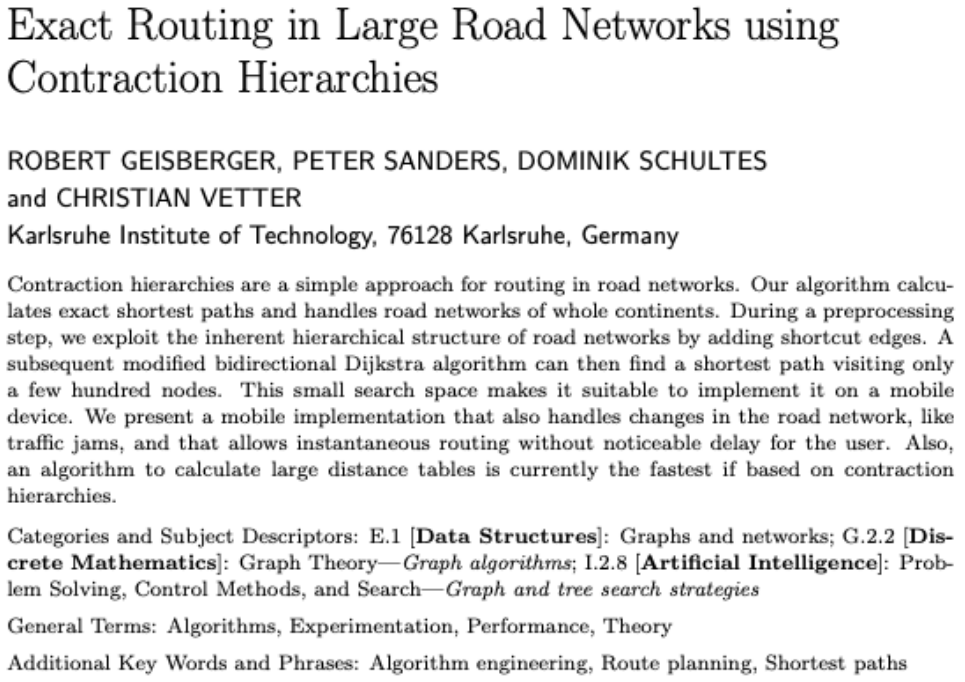
1. Contraction preserves shortest paths, and can only use nodes of higher rank.
2. Show: “Red edges” below all have to exist, by induction.
3. In particular: When v'_i is contracted, shortest path v'_{i+1} is preserved, using only nodes of higher rank than that of v_i.



Mandatory Assignment 3

- Implement contraction hierarchies and compare to bidirectional Dijkstra
- Much more open ended than other two assignments
- In groups of 3
- Due date Nov 12, 11:59

- Please follow code template on learnIT to have a good start



Exact Routing in Large Road Networks using Contraction Hierarchies

ROBERT GEISBERGER, PETER SANDERS, DOMINIK SCHULTES
and CHRISTIAN VETTER
Karlsruhe Institute of Technology, 76128 Karlsruhe, Germany

Contraction hierarchies are a simple approach for routing in road networks. Our algorithm calculates exact shortest paths and handles road networks of whole continents. During a preprocessing step, we exploit the inherent hierarchical structure of road networks by adding shortcut edges. A subsequent modified bidirectional Dijkstra algorithm can then find a shortest path visiting only a few hundred nodes. This small search space makes it suitable to implement it on a mobile device. We present a mobile implementation that also handles changes in the road network, like traffic jams, and that allows instantaneous routing without noticeable delay for the user. Also, an algorithm to calculate large distance tables is currently the fastest if based on contraction hierarchies.

Categories and Subject Descriptors: E.1 [**Data Structures**]: Graphs and networks; G.2.2 [**Discrete Mathematics**]: Graph Theory—*Graph algorithms*; I.2.8 [**Artificial Intelligence**]: Problem Solving, Control Methods, and Search—*Graph and tree search strategies*

General Terms: Algorithms, Experimentation, Performance, Theory

Additional Key Words and Phrases: Algorithm engineering, Route planning, Shortest paths

Paper on learnIT

Speed-up considerations

Contraction Hierarchies works for arbitrary ordering choices, but might be really slow during preprocessing!

- How to find contraction ordering?
  - Random?
  - Edge difference
    - How many edges are added/removed in contraction
    - Want to add as few as possible
- How to change the graph?
  - Expensive: Removing edges
  - Can we ignore edges/nodes?

- How to find out if (u, v, w) is unique, i.e., (u, w) has to be added a shortcut
  - Dijkstra from u to w ignoring v?
  - When should we stop?
  - How can we initialize a search efficiently? (Don't reset all n dist values!)
  - Fixed bound on number of settled nodes?
    - Might add "unnecessary shortcuts"

A real algorithmic beauty

- Improves a well-known algorithm
- Implementation not too difficult
- Fast running times on ”instances of practical relevance”
- Correctness not obvious